
Intro to Machine Learning

BA4 • 2025-2026

Contents

1	Introduction	5
1.1	ML Basics	5
1.1.1	Rappels mathématiques	6
2	Linear Supervised ML	7
2.1	Linear regression	7
2.1.1	Train	7
2.1.2	Test	9
2.1.3	Complement: multiple outputs	9
2.1.4	Model evaluation on test sample	10
2.2	Linear models for classification	11
2.2.1	Logistic Regression	11
2.2.2	Classification evaluation	14
2.2.3	Dealing with multi class	15
2.2.4	Evaluation metrics on multi-class classification	17
2.2.5	SVM – Support Vector Machines	18
3	Non-linear Supervised ML	21
3.1	k-Nearest Neighbors	21
3.1.1	Distances	21
3.1.2	Classification	22
3.1.3	Regression	22
3.2	Feature expansion	24
3.2.1	Polynomial curve fitting	24
3.2.2	Fitting and regularization – Cross validation	26
3.2.3	Penalizing overfitting – Regularization	27
3.3	Kernel methods	30
3.3.1	Kernelizing an algorithm	30
3.3.2	Kernelized linear regression	31
3.3.3	Kernelized ridge regression	31
3.3.4	How to find a good kernel ?	32
3.3.5	Multi-output kernelized ridge regression	32
3.4	Data representation	33
3.4.1	For text data — Bag of Words (BoW)	33
3.4.2	For image data — Visual Bag of Words	34

3.4.3	Data pre-processing	34
3.4.4	Data normalization	36
3.4.5	Imbalanced data - sampling methods	36
3.5	Artificial neural networks & MLP (deep learning)	40
3.5.1	Parametric feature expansion	40
3.5.2	Architecture	41
3.5.3	Activation functions	42
3.5.4	Multilayer perceptron (MLP)	42
3.5.5	Training – Backpropagation	42
3.5.6	Stochastic gradient descent	44
3.5.7	Lecture 10 — Deep Learning (Part 2)	45
4	Unsupervised ML	46
4.1	Dimensionality reduction	46
4.1.1	Definition	46
4.1.2	Example	46
4.2	Principal Component Analysis (PCA)	46
4.2.1	Definition	46
4.2.2	PCA Objective	46
4.2.3	1D PCA	47
4.2.4	Covariance Matrix	47
4.2.5	Optimization Problem	47
4.2.6	Higher Dimensional PCA	47
4.2.7	Limit Case: $d = D$	48
4.2.8	Keeping Non-zero Eigenvalues	48
4.2.9	Variance Retention	48
4.2.10	Reconstruction	48
4.2.11	PCA Mapping	48
4.3	Kernel PCA	48
4.3.1	Feature Mapping	48
4.3.2	Centered Feature Space	49
4.3.3	Representer Theorem	49
4.3.4	Kernel PCA Projection	49
4.3.5	Non-centered Data	49
4.3.6	Kernel PCA vs PCA	49
4.4	Autoencoders	50
4.4.1	PCA Linear Mapping	50
4.4.2	Simple Autoencoder	50
4.4.3	Autoencoder: General Form	50

4.4.4	Deep Autoencoder	50
4.4.5	Training	50
4.4.6	Dimensionality Expansion	50
4.4.7	Denoising Autoencoder	51
4.4.8	Sparse Autoencoder	51
4.4.9	Applications	51
4.4.10	Autoencoder Mapping	51
4.5	K-Means Clustering	52

CHAPTER 1

Introduction

1.1 ML Basics

Definitions

- **Data sample:** an individual observation.
- **Data set:** a collection of multiple data samples.
- **Regression:** task on continuous values.
- **Classification:** task on discrete labels.

Notations

$$x_i \in \mathbb{R}^D$$

It is a column vector of dimension D (the number of features).

$$y_i \in \mathbb{R}^C$$

It is the target output. For regression, $C = 1$ (a single real value). For multi-class classification with C classes, y_i is a one-hot vector of dimension C .

In matrix form, the dataset of N samples is written:

$$X \in \mathbb{R}^{N \times D}, \quad y \in \mathbb{R}^N$$

where row i of X is x_i^T .

1.1.1 Rappels mathématiques

Rappels – Dérivée et optimalité

Si on a $R(w)$ une fonction.

$$R'(\bar{w}) = \frac{dR}{dw}(\bar{w}) = \lim_{w \rightarrow 0} \frac{R(\bar{w} + w) - R(\bar{w})}{w}$$

Si

$$R'(\bar{w}) = 0$$

alors cela peut être :

- un minimum,
- un maximum,
- ou un point de selle.

On assumera dans la majorité des cas que la solution trouvée sera un minimum.

La solution à partir d'une telle équation, donc le w optimisé, sera dénoté

$$w^*$$

Gradient

Le gradient est la généralisation de la dérivée pour les fonctions multi-variables. Il pointe dans la direction de plus forte augmentation de R .

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w_1} \\ \frac{\partial R}{\partial w_2} \\ \vdots \\ \frac{\partial R}{\partial w_n} \end{pmatrix}$$

Si le gradient est égal au vecteur nul :

$$\nabla_w R(w) = 0$$

alors on obtient un *stationary point* lorsque l'on recherche w^* .

Useful Identities

$$\nabla_b(a^T b) = a$$

$$\|a\|^2 = a^T a$$

$$\nabla_a \|a\|^2 = 2a$$

$$\nabla_B(a^T B) = a$$

CHAPTER 2

Linear Supervised ML

2.1 Linear regression

Pourquoi et quand utiliser la régression linéaire ?

Pourquoi : La régression linéaire est le modèle le plus simple pour prédire une valeur continue. Elle a une solution analytique exacte (pas besoin d'itérations), est rapide à entraîner, et ses paramètres sont facilement interprétables.

Quand : On l'utilise lorsque la relation entre les features x_i et la cible y_i est approximativement linéaire, et que la sortie est une valeur continue (ex : prédire un prix, une température).

Output : Un scalaire $\hat{y}_i \in \mathbb{R}$ (ou un vecteur pour le multi-output).

Le principe de la régression linéaire est d'essayer de fitter une droite, un plan, ou un hyperplan sur les données que nous avons afin de pouvoir en faire des approximations.

2.1.1 Train

On cherche à trouver le vecteur w optimal tel que :

$$\hat{y}_i = w^T x_i$$

Pour trouver ce w , on cherche le w optimal noté w^* , tel que les prédictions soient les plus proches des vraies valeurs.

Measure of closeness

Pour mesurer la distance entre la prédiction et la réalité, on utilise la distance euclidienne :

$$d(\hat{y}_i, y_i) = \sqrt{(\hat{y}_i - y_i)^2}$$

En pratique, on utilise la squared Euclidean distance :

$$d^2(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

Comme la racine carrée est monotone croissante sur les réels positifs, minimiser la distance ou son carré revient au même. Utiliser le carré a aussi l'avantage de rendre la fonction différentiable et de pénaliser plus fortement les grandes erreurs.

On va alors chercher à minimiser cette distance, pour être le plus proche possible des vrais résultats. Le training devient donc un minimization problem où on va tenter de minimiser ce que l'on définit comme la fonction de coût (empirical risk) :

$$R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$$

Le problème de training devient donc :

$$\min_w R(w)$$

Empirical Risk

La **empirical risk** $R(w)$ mesure l'erreur moyenne du modèle sur les données d'entraînement. Sa forme générale est :

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

où ℓ est la *loss function* (fonction de perte). En régression linéaire, on utilise $\ell(\hat{y}, y) = (\hat{y} - y)^2$.

Pour minimiser $R(w)$, on calcule le gradient et on impose qu'il soit nul. On applique la dérivation terme à terme :

$$\nabla_w R(w) = \frac{1}{N} \sum_{i=1}^N \nabla_w ((w^T x_i - y_i)^2)$$

En utilisant la règle de la chaîne :

$$\nabla_w (u^2) = 2u \nabla_w u$$

avec $u = w^T x_i - y_i$, et en notant que $\nabla_w (w^T x_i) = x_i$, on obtient :

$$\nabla_w R(w) = \frac{2}{N} \sum_{i=1}^N (w^T x_i - y_i) x_i$$

On impose la condition d'optimalité :

$$\nabla_w R(w) = 0$$

Ce qui donne :

$$\sum_{i=1}^N x_i x_i^T w^* = \sum_{i=1}^N x_i y_i$$

Forme matricielle :

$$X^T X w^* = X^T y$$

Si $X^T X$ est inversible :

$$w^* = (X^T X)^{-1} X^T y$$

Sinon (cas sous-déterminé ou sur-déterminé), on utilise la pseudo-inverse :

$$w^* = X^\dagger y$$

2.1.2 Test

Une fois le modèle entraîné, on prédit sur un nouveau point x :

$$\hat{y} = w^{*T} x$$

Output : un scalaire $\hat{y} \in \mathbb{R}$.

2.1.3 Complement: multiple outputs

On généralise à plusieurs sorties ($C > 1$) :

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

avec :

$$W = [w_1 \ w_2 \ \dots \ w_C] \in \mathbb{R}^{D \times C}$$

Chaque colonne w_k prédit la k -ème sortie indépendamment. La solution devient :

$$W^* = X^\dagger Y$$

Conclusion :

La régression linéaire permet d'obtenir une solution fermée exacte grâce à l'optimisation d'une fonction de coût quadratique convexe. Cela garantit que le minimum trouvé correspond à l'optimum global.

Ainsi, une fois W^* calculé, la phase de test consiste simplement à appliquer :

$$\hat{y} = W^{*T} x$$

Le modèle est donc entièrement déterminé par la solution analytique du problème d'optimisation.

2.1.4 Model evaluation on test sample

Après training, il faut mesurer si le modèle est bon sur des données qu'il n'a pas vues. On utilise pour cela des métriques calculées sur le jeu de test.

Mean Squared Error (MSE)

Moyenne des erreurs au carré. Pénalise fortement les grandes erreurs.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

Racine carrée du MSE. Exprimée dans la même unité que y_i , donc plus interprétable.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE)

Moyenne des erreurs absolues. Moins sensible aux outliers que le MSE.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

Mean Absolute Percentage Error (MAPE)

Erreur relative en pourcentage. Utile pour comparer des modèles sur des cibles d'échelles différentes. Attention : ne fonctionne pas si $y_i = 0$.

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

2.2 Linear models for classification

2.2.1 Logistic Regression

Pourquoi et quand utiliser la logistic regression ?

Pourquoi : On ne peut pas utiliser directement la régression linéaire pour la classification car ses prédictions $w^T x_i$ peuvent prendre n'importe quelle valeur dans $\mathbb{R}$, alors qu'on cherche une probabilité dans $[0, 1]$. La logistic regression résout ce problème en passant la prédiction linéaire à travers une fonction sigmoid.

Quand : Problème de classification binaire (2 classes) avec des données approximativement linéairement séparables dans l'espace des features.

Output : Une probabilité $\hat{y}_i \in (0, 1)$ interprétée comme $p(\text{class} = 1 \mid x_i)$, puis un label $\{0, 1\}$ par seuillage.

Pourquoi pas la linear regression pour classifier ?

On pourrait utiliser la régression linéaire pour un problème de classification en seuillant le résultat :

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Mais cette méthode n'est pas fiable : les prédictions ne sont pas des probabilités, et la fonction de perte quadratique pénalise les points bien classifiés mais éloignés de la frontière.

Logistic Sigmoid Function

Pour obtenir une probabilité, on applique la **sigmoid function** à la combinaison linéaire $w^T x_i$:

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Elle mappe $\mathbb{R} \rightarrow (0, 1)$, ce qui permet d'interpréter la sortie comme une probabilité.

Dans notre cas, la prédiction devient :

$$\hat{y}_i = \sigma(w^T x_i) = \frac{1}{1 + \exp(-w^T x_i)}$$

Training

On veut trouver w^* qui maximise la vraisemblance des observations. Pour un sample i de label $y_i \in \{0, 1\}$, la probabilité du label donné w est :

$$p(y_i | x_i, w) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Sur tout le dataset (échantillons supposés indépendants) :

$$p(y|w) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

En prenant le log (pour transformer le produit en somme) et en passant à la minimisation (on minimise le négatif), on obtient la **binary cross-entropy loss** :

$$R(w) = - \sum_{i=1}^N \left(y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right)$$

Interprétation :

- Pour les samples positifs ($y_i = 1$) : on veut maximiser $\ln(\hat{y}_i)$, donc on pénalise si $\hat{y}_i$ est faible.
- Pour les samples négatifs ($y_i = 0$) : on veut maximiser $\ln(1 - \hat{y}_i)$, donc on pénalise si $\hat{y}_i$ est élevé.

Contrairement à la régression linéaire, il n'existe pas de solution fermée. On utilise donc le **gradient descent**.

Le gradient se calcule en séparant positifs et négatifs puis en les réunissant :

$$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$$

Gradient Descent Algorithm

But : minimiser $R(w)$ de façon itérative en descendant dans la direction opposée au gradient.

1. Initialize w_0 randomly
2. While not converged:

$$w_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$$

- $\eta > 0$ est le **learning rate** : trop grand $\Rightarrow$ divergence ; trop petit $\Rightarrow$ convergence lente.
- À chaque itération, on se rapproche du minimum de R .

Convergence criteria

On arrête lorsqu'une des conditions suivantes est satisfaite :

- $|R(w_k) - R(w_{k-1})| < \delta_R$ (la loss ne bouge plus)
- $\|w_k - w_{k-1}\| < \delta_w$ (les paramètres ne bougent plus)
- Nombre fixe d'itérations atteint (budget computationnel)

Test

Une fois w^* trouvé :

$$\hat{y}_i = \sigma(w^{*T} x_i)$$

Puis classification par seuillage :

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_i \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Le seuil 0.5 peut être ajusté selon le contexte (ex : priorité à la précision ou au recall).

2.2.2 Classification evaluation

Après entraînement, comment savoir si le modèle classe correctement ? On compare les labels prédits aux labels réels via une **confusion matrix** (pour 2 classes, c'est un tableau 2×2) :

Predicted class	True class	
	Pos	Neg
Pos	TP	FP
Neg	FN	TN

- **TP** (True Positive) : prédit positif, vrai label positif.
- **TN** (True Negative) : prédit négatif, vrai label négatif.
- **FP** (False Positive, type I error) : prédit positif, vrai label négatif.
- **FN** (False Negative, type II error) : prédit négatif, vrai label positif.

$$P = TP + FN \quad (\text{total real positives}) \quad N = FP + TN \quad (\text{total real negatives})$$

Accuracy

Definition: proportion of correctly classified samples (all classes combined).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Limitation : trompeuse sur des données déséquilibrées (ex : 99% de négatifs $\Rightarrow$ prédire toujours négatif donne 99% d'accuracy).

Precision

Definition: parmi tout ce qui a été prédit positif, quelle fraction l'est vraiment ?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Quand la prioriser : quand les fausses alarmes sont coûteuses (ex : spam filter).

Recall (Sensitivity)

Definition: parmi tous les vrais positifs, quelle fraction a été détectée ?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Quand le prioriser : quand manquer un positif est coûteux (ex : détection de cancer).

False Positive Rate (FPR)

Definition: proportion de négatifs incorrectement classifiés comme positifs.

$$\text{FPR} = \frac{FP}{FP + TN}$$

F1 Score

La précision et le recall sont souvent en tension (améliorer l'un dégrade l'autre). Le F1-score est leur moyenne harmonique :

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Interpretation: The F1-score ranges between 0 and 1.

- $F1 \approx 1$: excellent balance between precision and recall
- $F1 \approx 0.5$: imbalance between precision and recall
- $F1 \approx 0$: very poor model

Quand l'utiliser : données déséquilibrées où l'accuracy est trompeuse.

2.2.3 Dealing with multi class

Maintenant, ce qu'on a fait pour la classification marchait seulement si on devait choisir entre 2 classes. Mais comment fait-on lorsque l'on a $C > 2$ classes ?

On représente alors les labels différemment avec le **one-hot encoding** : y_i devient un vecteur de dimension C contenant des 0 et un unique 1 à la position correspondant au label.

Exemple pour $C = 7$ classes, label = classe 3 :

$$y_i = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Linear classification (multi-class)

La méthode de train devient un problème de régression multi-output :

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

dont la solution est :

$$W^* = X^\dagger Y$$

Pour prédire, on calcule un score pour chaque classe :

$$\hat{y}_i = W^{*T} x_i \in \mathbb{R}^C$$

On choisit la classe ayant le score le plus élevé :

$$k = \arg \max_j \hat{y}_i^{(j)}$$

Logistic regression (multi-class – Softmax)

On remplace la fonction sigmoid par la **Softmax** qui généralise sigmoid à C classes. La Softmax produit une distribution de probabilité sur toutes les classes (les valeurs sont dans $(0, 1)$ et leur somme est 1) :

$$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T} x)}{\sum_{j=1}^C \exp(w^{(j)T} x)}$$

Avec :

- $w^{(k)}$: la k -ème colonne de W (poids pour la classe k)
- C : le nombre de classes
- $\hat{y}^{(k)}(x) \approx p(\text{class} = k \mid x)$

L'empirical risk devient la **cross-entropy loss** multi-classe :

$$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln(\hat{y}_i^{(k)})$$

Et on applique le gradient descent avec :

$$\nabla_W R(W) = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T$$

Le résultat final est obtenu par :

$$k = \arg \max_j \hat{y}^{(j)}(x)$$

2.2.4 Evaluation metrics on multi-class classification

Avec plusieurs classes, on généralise la confusion matrix en un tableau $C \times C$.

Confusion matrix – 3 classes

	True A	True B	True C
Predicted A	...	...	...
Predicted B	...	...	...
Predicted C	...	...	...

La diagonale contient les bons classements. Les éléments hors-diagonale sont des erreurs.

L'Accuracy se généralise directement :

$$\text{Accuracy} = \frac{\sum_{k=1}^C \text{true positives for class } k}{\text{total samples}}$$

Pour Precision, Recall, FPR et F1-score, on calcule la métrique pour chaque classe (one-vs-all) puis on en fait la moyenne.

Macro vs Weighted Average

- **Macro average** : moyenne simple sur toutes les classes. Chaque classe contribue également, quelle que soit sa taille.
- **Weighted average** : moyenne pondérée par le nombre de samples par classe. Prend en compte le déséquilibre des classes (*class imbalance*).

En pratique, si les classes sont déséquilibrées, préférer le **weighted average** ou l'analyse par classe séparément.

2.2.5 SVM – Support Vector Machines

Pourquoi et quand utiliser les SVM ?

Pourquoi : La logistic regression trouve *une* frontière qui sépare les classes, mais pas nécessairement la *meilleure*. Les SVM cherchent la frontière qui maximise la marge (distance aux points les plus proches), ce qui améliore la généralisation.

Quand : Classification binaire, surtout quand les données sont linéairement séparables ou presque. Très efficace en haute dimension. S'étend aux problèmes non-linéaires via les kernels.

Output : Un label binaire $\hat{y} \in \{-1, +1\}$.

Intuition

On cherche un hyperplan $w^T x + b = 0$ qui sépare les deux classes ($y_i \in \{-1, +1\}$). Parmi tous les hyperplans qui séparent correctement les données, on choisit celui qui maximise la marge.

Margin

La marge est la distance minimale entre l'hyperplan et les points les plus proches de chaque classe.

$$\text{margin} = \frac{2}{\|w\|}$$

Les points qui définissent la marge (les plus proches de l'hyperplan) sont appelés les **support vectors**.

Hard-margin SVM (données linéairement séparables)

On maximise la marge sous contrainte que tous les points soient correctement classifiés :

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 \quad \forall i$$

La contrainte $y_i(w^T x_i + b) \geq 1$ force chaque point à être du bon côté de la marge.

Soft-margin SVM (données non parfaitement séparables)

En pratique, les données ne sont pas toujours linéairement séparables. On introduit des **slack variables** $\xi_i \geq 0$ qui permettent à certains points de violer la marge :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Avec :

- $C > 0$: hyperparamètre contrôlant le compromis entre marge large et erreurs tolérées.
- C grand $\Rightarrow$ peu d'erreurs tolérées (risque d'overfitting).
- C petit $\Rightarrow$ marge plus large mais plus d'erreurs (risque d'underfitting).

Prédiction

$$\hat{y} = \text{sign}(w^T x + b) \in \{-1, +1\}$$

Regularization built-in

Le terme $\frac{1}{2} \|w\|^2$ dans l'objectif joue le rôle d'une régularisation L2, qui pénalise les poids trop grands. C'est pourquoi les SVM n'ont pas besoin d'un terme de régularisation additionnel explicite.

Kernel SVM

Pour des données non-linéairement séparables, on peut kerneliser le SVM en remplaçant le produit scalaire $x_i^T x_j$ par un kernel $k(x_i, x_j)$ dans la formulation duale. Cela revient à travailler dans un espace de features de grande dimension sans le calculer explicitement.

CHAPTER 3

Non-linear Supervised ML

3.1 k-Nearest Neighbors

Pourquoi et quand utiliser KNN ?

Pourquoi : KNN est un algorithme non-paramétrique : il ne fait aucune hypothèse sur la forme des données. Il mémorise l'ensemble du training set et prédit en cherchant les points les plus similaires.

Quand : Données avec des frontières de décision complexes et non-linéaires. Fonctionne bien sur de petits datasets. Simple à implémenter et sans phase d'entraînement.

Limitations : Prédiction lente (distance à tous les points de training à chaque fois). Sensible aux features mal scalées et aux outliers. Pas adapté aux grands datasets.

Output : Un label de classe (classification) ou une valeur continue (régression).

3.1.1 Distances

Pour déterminer les voisins les plus proches, on choisit une mesure de distance. Le choix de la distance dépend de la nature des données.

Euclidian distance

Mesure standard de la distance géométrique entre deux points. Adaptée aux features continues de même échelle.

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^D (x_i^{(k)} - x_j^{(k)})^2}$$

Chi-square distance

Distance souvent utilisée pour comparer des distributions ou histogrammes (ex : features de fréquence).

$$d(x_i, x_j) = \sqrt{\chi^2(x_i, x_j)} = \sqrt{\sum_{k=1}^D \frac{(x_i^{(k)} - x_j^{(k)})^2}{x_i^{(k)} + x_j^{(k)}}}$$

3.1.2 Classification

Pour la classification : on prend le vote majoritaire parmi les k voisins les plus proches.

KNN classification algorithm

1. Compute distance between x and all x_i in the training set
2. Find the k samples with minimum distances
3. Find the most common label among these k nearest neighbors (majority vote)
4. Assign that label as $\hat{y}$

Choix de k

- $k = 1$: modèle très flexible, risque d'overfitting.
- k grand : modèle plus lisse, risque d'underfitting.
- On choisit k par cross-validation.
- En pratique, choisir k impair pour éviter les égalités (pour 2 classes).

3.1.3 Regression

Pour la régression : on fait la moyenne (éventuellement pondérée) des valeurs des k voisins.

KNN regression algorithm

1. Compute distance between x and all x_i
2. Find k samples with minimum distances
3. Compute the predicted value as a (weighted) average of the neighbors:

$$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}$$

avec $w_i = 1$ (unweighted) ou $w_i = \frac{1}{d_i}$ (inverse distance weighting : les voisins plus proches ont plus d'influence).

3.2 Feature expansion

3.2.1 Polynomial curve fitting

Pourquoi et quand utiliser la feature expansion ?

Pourquoi : Les modèles linéaires ne peuvent capturer que des relations linéaires entre les features et la cible. Si les données suivent une courbe, un cercle, ou un polynôme complexe, un modèle linéaire sera trop limité. La feature expansion transforme les features pour rendre les relations non-linéaires linéairement représentables.

Quand : Les données ont une structure non-linéaire dans l'espace original, mais on veut garder la simplicité et la tractabilité des modèles linéaires.

Output : Le même type de sortie que le modèle linéaire sous-jacent, mais avec une frontière de décision non-linéaire dans l'espace original.

Principe

On prend nos données de base x puis on leur applique une transformation ϕ qui crée un nouveau vecteur contenant différentes combinaisons (polynomiales) des features initiales.

Par exemple :

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Après polynomial curve expansion (degré 2) :

$$\phi(x) = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1x_2 \\ x_2^2 \end{pmatrix}$$

On obtient alors un modèle encore linéaire *dans l'espace transformé*, mais non-linéaire dans l'espace original :

$$\hat{y}_i = w^T \phi(x_i)$$

Avec :

- $\phi(x_i) \in \mathbb{R}^F$ (où F est le nombre de features après expansion)
- $w \in \mathbb{R}^F$

Solution analytique

On obtient alors le gradient :

$$\nabla_w R(w) = 2 \sum_{i=1}^N \phi(x_i) \left(\phi(x_i)^T w - y_i \right)$$

En imposant $\nabla_w R(w) = 0$, on obtient :

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

où $\Phi \in \mathbb{R}^{N \times F}$ est la matrice dont la ligne i est $\phi(x_i)^T$.

Cover's theorem

A complex pattern classification problem cast in a high-dimensional space non-linearly is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated.

En d'autres termes : en augmentant la dimensionnalité via ϕ , on facilite la séparation linéaire des classes.

3.2.2 Fitting and regularization – Cross validation

Definition: Overfitting

Le modèle mémorise les données d'entraînement mais ne généralise pas.

Symptôme : erreur faible sur training, erreur grande sur test.

Cause typique : modèle trop complexe (ex : polynôme de trop haut degré).

Definition: Underfitting

Le modèle est trop simple pour capturer la structure des données.

Symptôme : erreur grande sur training *et* test.

Cause typique : modèle pas assez expressif.

Pourquoi la cross-validation ?

On ne peut pas utiliser les données de test pour choisir le modèle (cela biaiserait l'évaluation). La cross-validation simule le comportement sur des données non vues en utilisant uniquement les données d'entraînement. Elle permet de choisir les hyperparamètres (ex : degré du polynôme, k dans KNN) et d'estimer l'erreur de généralisation.

Validation set

On sépare le dataset en train/validation avant d'entraîner, puis on évalue sur la partie validation.

Ratios courants :

80/20 ou 70/30

Avantage : rapide. **Inconvénient** : l'estimation dépend du split choisi aléatoirement.

Leave-one-out cross validation (LOOCV)

For $i = 1 \dots N$

1. Temporarily remove (x_i, y_i) from the training set
2. Train on the remaining $N - 1$ samples
3. Compute the error on (x_i, y_i)

Report the average error over all N folds.

Avantage : utilise presque toutes les données pour l'entraînement. **Inconvénient** : N entraînements = très coûteux.

K-fold cross validation

On divise les données en K partitions (folds).

Pour chaque fold $k = 1, \dots, K$:

- Entraîner sur les $K - 1$ autres folds
- Tester sur le fold k
- Calculer l'erreur

On reporte la moyenne des erreurs sur les K folds.

Avantage : bon compromis coût/fiabilité. **Valeur typique** : $K = 5$ ou $K = 10$.

Comparison

- **Validation set** : rapide, mais estimation instable (dépend du split)
- **LOOCV** : très coûteux, estimation quasi-optimale
- **K-fold** : meilleur compromis vitesse/fiabilité

3.2.3 Penalizing overfitting – Regularization*Pourquoi régulariser ?*

Avec la polynomial curve expansion, on peut avoir $F \gg N$ features (plus de features que de samples), ce qui conduit à de l'overfitting. La régularisation ajoute une pénalité sur la norme de w pour forcer les poids à rester petits, ce qui simplifie le modèle et améliore la généralisation.

Nouvelle training objective :

$$E(w) = \underbrace{R(w)}_{\text{fit to data}} + \underbrace{\lambda E_w(w)}_{\text{regularizer}}$$

Avec :

- $\lambda > 0$: hyperparamètre contrôlant l'influence du régularisateur (choisi par cross-validation)
- $E_w(w)$: terme de pénalité sur la complexité du modèle

L2 regularization – Ridge Regression

L2 regularization

$$E_w(w) = w^T w = \|w\|^2$$

Pénalise les poids de grande norme. Tend à distribuer les poids uniformément (pas de mise à zéro).

Le problème devient :

$$\min_w \sum_{i=1}^N \left(\phi(x_i)^T w - y_i \right)^2 + \lambda w^T w$$

Le gradient :

$$\nabla_w E(w) = 2 \sum_{i=1}^N \phi(x_i) \left(\phi(x_i)^T w - y_i \right) + 2\lambda w$$

La solution analytique devient (toujours une forme fermée !) :

$$w^* = \left(\Phi^T \Phi + \lambda I_F \right)^{-1} \Phi^T y$$

Ceci est la **Ridge Regression**. Ajouter λI_F rend la matrice toujours inversible, même si $\Phi^T \Phi$ ne l'est pas.

Multi-output ridge regression

Sur des problèmes multi-output :

$$\min_W \sum_{i=1}^N \|W^T \phi(x_i) - y_i\|^2 + \lambda \|W\|_F^2$$

Avec $\|W\|_F$ la norme de Frobenius ($\sqrt{\sum_{i,j} W_{ij}^2}$).

Solution :

$$W^{\star} = \left(\Phi^T \Phi + \lambda I_F \right)^{-1} \Phi^T Y$$

3.3 Kernel methods

Pourquoi et quand utiliser les kernels ?

Problème avec la feature expansion : si on veut des features polynomiales de degré élevé, F devient exponentiellement grand (ex : $D = 10$ features, degré 3 $\Rightarrow$ des milliers de features). Le stockage et le calcul de $\phi(x)$ deviennent infaisables.

Solution – le kernel trick : on n'a jamais besoin de $\phi(x)$ explicitement ! Il suffit de calculer des produits scalaires $\phi(x_i)^T \phi(x_j)$. On définit directement une **fonction de similarité** $k(x_i, x_j)$ qui calcule ce produit scalaire implicitement.

Quand : données avec structure non-linéaire complexe ; $F \gg N$; quand choisir ϕ explicitement est difficile.

Output : même type de sortie que le modèle de base (régression ou classification), avec une frontière non-linéaire dans l'espace original.

Plutôt que de définir ϕ explicitement, on définit une fonction de similarité :

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad (\text{même si } \phi \text{ peut être inconnu ou de dimension infinie})$$

k est un scalaire (pas un vecteur).

Exemples de kernels

Polynomial kernel (correspond à une expansion polynomiale) :

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

Gaussian (RBF) kernel (correspond à un espace de features de dimension *infinie*) :

$$k(x_i, x_j) = \exp \left(-\frac{\|x_i - x_j\|^2}{2\sigma^2} \right)$$

Plus x_i et x_j sont proches, plus $k(x_i, x_j)$ est grand (proche de 1). Plus ils sont loin, plus k est petit (proche de 0).

3.3.1 Kernelizing an algorithm

Puisqu'un kernel correspond à un produit scalaire dans l'espace des features, on peut remplacer tout produit scalaire $\phi(x_i)^T \phi(x_j)$ par $k(x_i, x_j)$ dans les algorithmes existants.

Kernel vector (similarité entre un nouveau point x_i et tous les points du training set) :

$$k(X, x_i) = \begin{pmatrix} k(x_1, x_i) \\ k(x_2, x_i) \\ \vdots \\ k(x_N, x_i) \end{pmatrix} \in \mathbb{R}^N$$

Kernel matrix (toutes les similarités entre training points) :

$$K = \begin{pmatrix} k(x_1, x_1) & \cdots & k(x_1, x_N) \\ \vdots & \ddots & \vdots \\ k(x_N, x_1) & \cdots & k(x_N, x_N) \end{pmatrix} \in \mathbb{R}^{N \times N}$$

K est symétrique et semi-définie positive.

3.3.2 Kernelized linear regression

Normalement :

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

Avec kernels : on reparamétrise $w = \Phi^T \alpha$ (le vecteur w s'exprime comme combinaison linéaire des $\phi(x_i)$). Le problème s'exprime en $\alpha \in \mathbb{R}^N$:

$$\alpha^* = K^{-1} y$$

Prédiction finale (sans jamais calculer ϕ) :

$$\hat{y} = w^{*T} \phi(x) = y^T K^{-1} k(X, x)$$

3.3.3 Kernelized ridge regression

$$E(\alpha) = \sum (\alpha^T k(X, x_i) - y_i)^2 + \lambda \alpha^T K \alpha$$

Solution :

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

Prédiction :

$$\hat{y} = y^T (K + \lambda I_N)^{-1} k(X, x)$$

3.3.4 How to find a good kernel ?

Par **cross-validation** : on teste différents kernels (polynomial, RBF...) et leurs hyperparamètres ($d, \sigma \dots$) et on choisit celui qui minimise l'erreur de validation.

3.3.5 Multi-output kernelized ridge regression

$$W^* = \Phi^T (K + \lambda I)^{-1} Y$$

Prédiction :

$$\hat{y} = W^{*T} \phi(x) = Y^T (K + \lambda I)^{-1} k(X, x)$$

3.4 Data representation

Les algorithmes de ML attendent des vecteurs numériques $x_i \in \mathbb{R}^D$. Mais les données brutes peuvent être du texte, des images, des sons... Il faut donc les **représenter** sous forme vectorielle avant de les passer au modèle.

3.4.1 For text data — Bag of Words (BoW)

Pourquoi et quand utiliser le Bag of Words ?

Pourquoi : Les données textuelles sont des séquences de mots, mais les modèles de ML attendent des vecteurs de features numériques. Le BoW transforme un document en un vecteur de fréquences de mots, permettant d'appliquer des algorithmes classiques.

Quand : Classification de texte, détection de spam, analyse de sentiment — partout où la présence/fréquence des mots suffit à caractériser le document.

Output : $x_i \in \mathbb{R}^D$ où D est la taille du dictionnaire.

Bag of Words — procédure

1. Collecter tous les documents (corpus).
2. Construire un dictionnaire de D mots distincts.
3. Pour chaque document, compter les occurrences de chaque mot $\Rightarrow$ vecteur de comptage $\tilde{x}_i \in \mathbb{N}^D$.
4. Normaliser : diviser par le nombre total de mots du document $\Rightarrow$ vecteur de fréquences $x_i \in \mathbb{R}^D$.

Exemple BoW

Documents :

1. *John likes to watch movies. Mary likes movies too.*
2. *John also likes to watch football games.*

Dictionnaire ($D = 10$) : {John, likes, to, watch, movies, Mary, too, also, football, games}

Vecteurs de comptage $\tilde{x}$:

	John	likes	to	watch	movies	Mary	too	also	football	games
Doc 1	1	2	1	1	2	1	1	0	0	0
Doc 2	1	1	1	1	0	0	0	1	1	1

Représentation finale (normalisée par nombre de mots) :

$$x_1 = \frac{1}{9}[1, 2, 1, 1, 2, 1, 1, 0, 0, 0] \approx [0.11, 0.22, 0.11, \dots]$$

$$x_2 = \frac{1}{7}[1, 1, 1, 1, 0, 0, 0, 1, 1, 1] \approx [0.14, 0.14, 0.14, \dots]$$

Limitations du BoW

- **Pas d'ordre des mots** : “John likes Mary” et “Mary likes John” donnent le même vecteur.
- **Vocabulaire large** : D peut être très grand (dizaines de milliers de mots) $\Rightarrow$ vecteurs creux (sparse).
- **Mots fréquents non informatifs** : “the”, “is”, “a”... dominent les comptages. On les élimine souvent avec une liste de *stop words*.

3.4.2 For image data — Visual Bag of Words*Pourquoi et quand utiliser le Visual BoW ?*

Pourquoi : Une image brute est une grille de pixels — pas directement exploitable comme vecteur de features sémantiques. L'idée est d'appliquer le principe du BoW au contenu visuel : on extrait des petits morceaux de l'image (*patches*), on les regroupe en catégories visuelles (*visual words*), et on représente l'image par la distribution de ces catégories.

Quand : Classification d'images avec des méthodes classiques (SVM, KNN) avant l'ère deep learning.

Output : $x_i \in \mathbb{R}^D$ où D est la taille du dictionnaire visuel.

Visual Bag of Words — procédure

1. **Extraire des patches** : découper toutes les images en petites régions locales (ex : 16×16 pixels). Exemples : yeux, nez, bouches, textures...
2. **Construire un dictionnaire visuel** : regrouper tous les patches en D clusters.
3. **Encoder chaque image** : pour chaque patch d'une image,
 - trouver le visual word le plus proche dans le dictionnaire,
 - incrémenter de +1 l'entrée correspondante dans le vecteur BoW.
4. **Normaliser** : diviser par le nombre total de patches $\Rightarrow$ vecteur de fréquences $x_i \in \mathbb{R}^D$.

Discussion

We have seen that text and images can be represented as Bags of Words. What other type of data can you represent in this way? How would you do it?

Large and heterogeneous data can be made compact and homogeneous via BoW. The resulting representations can be directly used as input to any ML algorithm.

3.4.3 Data pre-processing

Raw data can be noisy, incomplete, or incorrectly formatted. Pre-processing improves data quality and model performance. Two main problems: **missing data** and **noise**.

Missing data

When we know that some data is missing, potential solutions:

Missing data — potential solutions

Strategy	Remark
Ignore the corresponding sample	Simple but loses information
Fill with a global constant	Fast, imprecise
Fill with the mean of the feature	Preserves global distribution
Fill with the mean of the feature for the same class	More precise if classes are distinct

Noise — incorrect values

Noise refers to incorrect or corrupted values in the data. A classical smoothing technique is **binning**.

Binning

For each attribute:

1. Sort the data.
2. Partition it into bins — either **equal width** or **equal frequency**.
3. Smooth the data by replacing each value with the **mean** or **median** of its bin.

Binning example

Data (sorted): 0, 4, 12, 16, 16, 18, 24, 26, 28

Equal width (width = 10):

Bin 1 $[-, 10)$	0, 4	mean = 2
Bin 2 $[10, 20)$	12, 16, 16, 18	mean = 15.5
Bin 3 $[20, +)$	24, 26, 28	mean = 26

Equal frequency (3 values per bin):

Bin 1	0, 4, 12	mean = 5.3
Bin 2	16, 16, 18	mean = 16.7
Bin 3	24, 26, 28	mean = 26

Clustering to detect outliers

Detect outliers as the clusters with too few elements and remove those samples.

Discussion

What other solutions to cleaning noisy data can you think of?

Use human knowledge to identify and correct errors (e.g. impossible values, typos...)

3.4.4 Data normalization

Goal: scale each individual attribute/feature dimension to fall within a specified range. This ensures no single feature dominates due to its scale.

Normalization methods

Method	Formula	Result range
Min-max	$x_i^{(d)} \leftarrow \frac{x_i^{(d)} - \min_d}{\max_d - \min_d}$	$[0, 1]$
Z-score	$x_i^{(d)} \leftarrow \frac{x_i^{(d)} - \mu_d}{\sigma_d}$	mean 0, std 1
Max normalization	$x_i^{(d)} \leftarrow \frac{x_i^{(d)}}{\max_d}$	$[-1, 1]$
Decimal scaling	$x_i^{(d)} \leftarrow \frac{x_i^{(d)}}{10^j}$	$\max x^{(d)} < 1$

where μ_d and σ_d are the mean and standard deviation of feature d across the dataset, and j is the smallest integer such that $\max(|x_i^{(d)}|) < 1$.

3.4.5 Imbalanced data - sampling methods

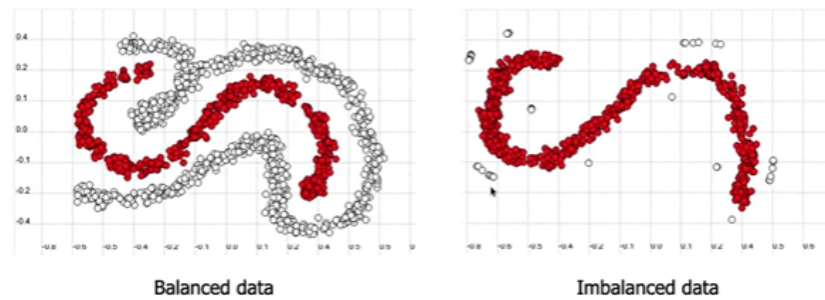
You train a classifier to predict if a subject suffers from cancer or not from image data. On validation data, your classifier gives you 99.1% accuracy. This sounds fantastic! We are ready to deploy it in the real world, right?

Out of curiosity, you check the confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	990	0
Actual Positive	9	1

You missed almost all the cancerous cases! So this is the problem of **imbalanced vs balanced**

data.



Two main approaches:

1. Act on the data $\Rightarrow$ **Sampling methods**
2. Act on the cost function $\Rightarrow$ **Cost-sensitive methods**

Sampling methods

Compensate the lack of data by rebalancing the classes:

Oversampling — increase the minority class

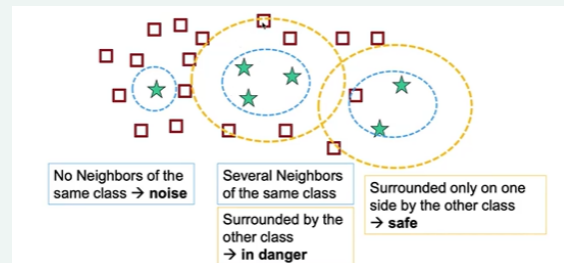
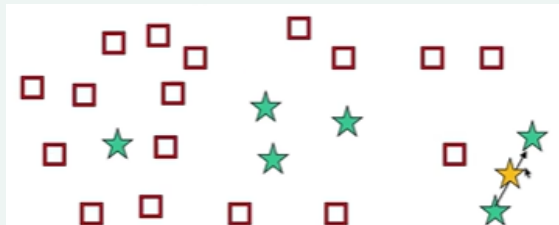
What is it? Generate new data inputs for the smallest class.

Naive: replicate data samples from the smallest class until balanced.

Drawback: may lead to overfitting.

Smarter: create new data samples based on neighboring existing ones (e.g. SMOTE).

Drawback: may create noisy samples.



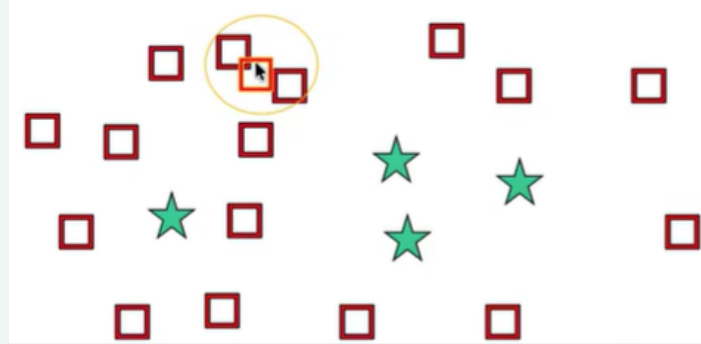
Undersampling — reduce the majority class

What is it? Remove redundant datapoints from the largest class.

Naïve: randomly remove samples from the largest class until balanced.

Drawback: may remove important samples.

Smarter: remove only the redundant samples. Works well if the majority class is densely populated.

*Cost-sensitive methods*

Given N training samples $\{(x_i, y_i)\}_{i=1}^N$, the standard empirical risk is:

$$R(\mathbf{x}_i, y_i, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

A simple solution consists of **re-weighting** the samples to put more emphasis on the rare class:

$$R(\mathbf{x}_i, y_i, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \beta_i \ell(\hat{y}_i, y_i)$$

where β_i is the weight of sample i , set **manually** (it cannot be optimized, as that would trivially give zero loss). It is chosen inversely proportional to the class frequency:

$$\beta_i = 1 - \frac{N_k}{N}$$

if sample i belongs to class k , which contains N_k samples (smaller class $\Rightarrow$ larger weight).

This re-weighting can be applied to any algorithm:

Weighted empirical risk — applications**Least-squares classification:**

$$\min_{\mathbf{W}} \sum_{i=1}^N \beta_i \|\mathbf{W}^T \mathbf{x}_i - \mathbf{y}_i\|^2 + \lambda \|\mathbf{W}\|_F^2$$

Logistic regression:

$$\min_{\mathbf{w}} - \sum_{i=1}^N \beta_i \sum_{k=1}^C y_i^{(k)} \ln(\mathbf{w}^T \mathbf{x}_i)$$

SVM (soft-margin):

$$\min_{w^{(0)}, \tilde{\mathbf{w}}} \frac{1}{2C} \|\tilde{\mathbf{w}}\|^2 + \sum_{i=1}^N \beta_i \max(0, 1 - y_i \cdot (w^{(0)} + \tilde{\mathbf{w}}^T \mathbf{x}_i))$$

3.5 Artificial neural networks & MLP (deep learning)

Pourquoi et quand utiliser les réseaux de neurones ?

Pourquoi : Les méthodes précédentes nécessitent de définir manuellement ϕ ou k . Les réseaux de neurones **apprennent automatiquement** une représentation des données adaptée au problème, couche par couche.

Quand : Grands datasets ; problèmes complexes avec des structures non-linéaires riches (images, texte, audio). En pratique, ils surpassent les méthodes précédentes dès que N est suffisamment grand.

Output : Tout type de sortie selon la couche finale : $\hat{y} \in \mathbb{R}$ (régression), $\hat{y} \in (0, 1)$ (classification binaire), distribution sur C classes (classification multi-classe).

3.5.1 Parametric feature expansion

Plutôt que de définir manuellement ϕ (comme dans le polynomial feature expansion), on va **apprendre** la meilleure expansion possible. On applique une transformation linéaire puis une non-linéarité, et on laisse le modèle optimiser les paramètres de cette transformation.

On définit la pré-activation de la première couche cachée :

$$a_{(1)} = W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix} = \begin{pmatrix} W_{(1)}^{(1,0)} + W_{(1)}^{(1,1)}x \\ \vdots \\ W_{(1)}^{(M,0)} + W_{(1)}^{(M,1)}x \end{pmatrix} = \begin{pmatrix} a_{(1)}^{(1)} \\ \vdots \\ a_{(1)}^{(M)} \end{pmatrix} \in \mathbb{R}^{M \times 1}$$

La **parametric feature expansion** consiste à appliquer une fonction d'activation f sur chaque pré-activation :

$$x \rightarrow z_{(1)} = \begin{bmatrix} f(a_{(1)}^{(1)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{bmatrix} = f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right) \in \mathbb{R}^{M \times 1}$$

Avec :

- f : fonction d'activation (ex: sigmoid, tangente hyperbolique, ReLU...)
- M : nombre d'unités cachées (dimension de la représentation interne)
- $W_{(1)}$: poids de la première couche (paramètres de la feature expansion)
- $z_{(1)}$: représentation cachée (output de la feature expansion)

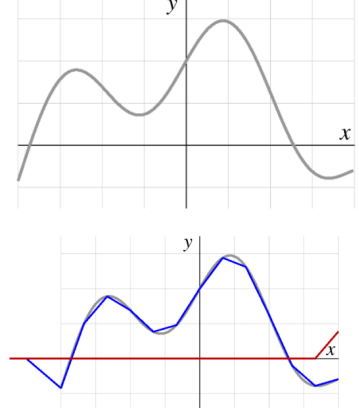
La prédiction finale est une combinaison linéaire des features apprises :

$$\hat{y} = w_{(2)}^T z_{(1)} = \sum_{i=1}^M w_{(2)}^{(i)} f\left(W_{(1)}^{(i,0)} + W_{(1)}^{(i,1)}x\right)$$

Universal approximation

We can approximate any continuous function with a linear combination of translated/scaled ReLU functions $f(\cdot)$

$$\begin{aligned}\hat{y} &= w_{(2)}^T z_{(1)} = \sum_{i=1}^M w_{(2)}^{(i)} f\left(W_{(1)}^{(i,0)} + W_{(1)}^{(i,1)} x\right) \\ &= w_{(2)}^{(1)} f\left(W_{(1)}^{(1,0)} + W_{(1)}^{(1,1)} x\right) \\ &\quad + w_{(2)}^{(2)} f\left(W_{(1)}^{(2,0)} + W_{(1)}^{(2,1)} x\right) \\ &\quad + w_{(2)}^{(3)} f\left(W_{(1)}^{(3,0)} + W_{(1)}^{(3,1)} x\right) + \dots\end{aligned}$$



3.5.2 Architecture

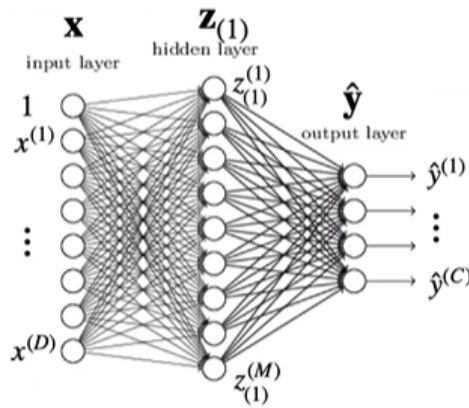
Un réseau de neurones est composé de couches successives de transformations affines suivies de non-linéarités.

Pour plusieurs features ($D > 1$) :

$$a_{(1)} = W_{(1)}^T \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^{M \times 1} \quad \text{and} \quad z_{(1)} = f(a_{(1)}) \in \mathbb{R}^{M \times 1}$$

Pour plusieurs sorties ($C > 1$), on remplace $w_{(2)}$ par une matrice $W_{(2)} \in \mathbb{R}^{M \times C}$:

$$\hat{y} = W_{(2)}^T z_{(1)} \in \mathbb{R}^{C \times 1}$$



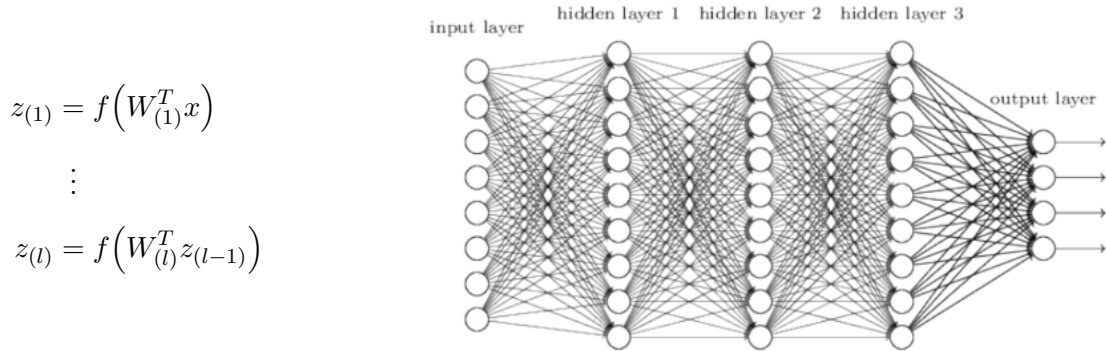
3.5.3 Activation functions

Fonctions d'activation courantes

- **Sigmoid** : $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$ – utilisée pour les sorties binaires.
- **Softmax** : $\sigma(z)^{(k)} = \frac{e^{z^{(k)}}}{\sum_j e^{z^{(j)}}}$ – utilisée en sortie pour la classification multi-classe.
- **ReLU** : $\sigma(z) = \max(0, z)$ – la plus utilisée dans les couches cachées.

3.5.4 Multilayer perceptron (MLP)

En ajoutant des couches intermédiaires, le modèle apprend des représentations de plus en plus abstraites :



$$\begin{aligned} z_{(1)} &= f(W_{(1)}^T x) \\ &\vdots \\ z_{(l)} &= f(W_{(l)}^T z_{(l-1)}) \end{aligned}$$

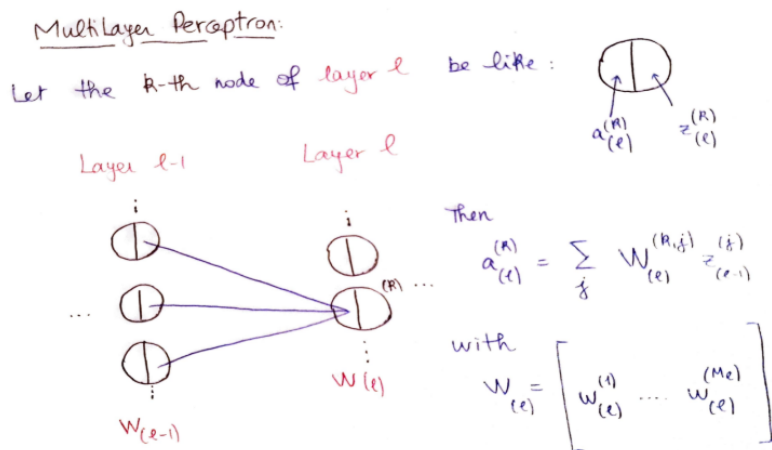
We call going from x to $\hat{y}$ the **forward pass** of the network:

$$\hat{y} = W_{(L)}^T z_{(L-1)} = W_{(L)}^T f(W_{(L-1)}^T f(W_{(L-2)}^T (\cdots f(W_{(1)}^T x) \cdots)))$$

- It is easier to simply think of $\hat{y}$ as a function of all parameters:

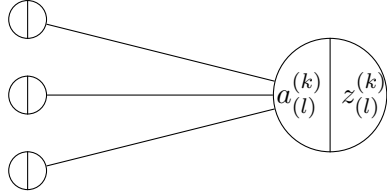
$$\hat{y} = \hat{y}(\mathbf{W}), \quad \mathbf{W} = \{W_{(l)}\}$$

3.5.5 Training – Backpropagation



La phase d'entraînement consiste à minimiser la loss function $R(\mathbf{W})$ par rapport à tous les paramètres $\{W_{(l)}\}$:

- **Régression** : $R(\mathbf{W}) = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$
- **Classification** : $R(\mathbf{W}) = -\frac{1}{N} \sum_i \sum_k y_i^{(k)} \ln \hat{y}_i^{(k)}$ (cross-entropy)

 Layer $l - 1$

 Layer l

$$a_{(l)}^{(k)} = \sum_j W_{(l)}^{(k,j)} z_{(l-1)}^{(j)}$$

$$z_{(l)}^{(k)} = f(a_{(l)}^{(k)})$$

$$W_{(l)} = \begin{bmatrix} w_{(l)}^{(1)} & w_{(l)}^{(2)} & \dots & w_{(l)}^{(M_l)} \end{bmatrix}$$

$$a_{(l)}^{(k)} = \left(w_{(l)}^{(k)} \right)^T z_{(l-1)}$$

Puisque le risque empirique est une moyenne, on peut se concentrer sur le gradient d'un seul terme ℓ_i :

$$R(\{W_{(l)}\}) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i) = \frac{1}{N} \sum_{i=1}^N \ell_i$$

On définit la sensibilité de la loss à la pré-activation du neurone (k, l) avec ℓ_i la loss du sample i :

$$\delta_{(l)}^{(k)} = \frac{\partial \ell_i}{\partial a_{(l)}^{(k)}}$$

Puisque $a_{(l)}^{(k)}$ dépend linéairement des poids :

$$\frac{\partial a_{(l)}^{(k)}}{\partial W_{(l)}^{(k,j)}} = z_{(l-1)}^{(j)}$$

Et par la règle de la chaîne :

$$\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

Le gradient $\nabla_{W^{(l)}} R$ est calculé par **backpropagation** en propageant l'erreur depuis la sortie vers l'entrée. On met ensuite à jour les paramètres (ou une variante comme **Adam**) :

$$W^{(l)} \leftarrow W^{(l)} - \eta \nabla_{W^{(l)}} R$$

VIDEO INCROYABLE: <https://www.youtube.com/watch?v=tIeHLnjs5U8>

Back propagation

1. Propagate x_i forward through the network (forward pass)
2. Compute $\delta^{(L)}$ depending on the loss
3. Propagate that delta backward to obtain each $\delta^{(l)}$
4. At each layer, compute the partial derivatives:

$$\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

3.5.6 Stochastic gradient descent

Vidéo qui explique très bien [3 mins]: <https://www.youtube.com/watch?v=UmathvAKj80>

Plutôt que de calculer le gradient sur tout le dataset (coûteux), on estime le gradient sur un **mini-batch** de B samples à chaque itération :

$$W_k \leftarrow W_{k-1} - \eta \nabla \ell_{i(k)}(W_{k-1})$$

$$W_k \leftarrow W_{k-1} - \eta \sum_{b=1}^B \nabla \ell_{(b,k)}(W_{k-1})$$

SGD avec momentum – on ajoute de l’inertie aux mises à jour pour accélérer la convergence :

$$\nu \leftarrow \gamma \nu + \eta \nabla R(W)$$

$$W \leftarrow W - \nu$$

avec $\gamma > 0$: accélère si le gradient ne change pas trop.

Pour la classification, on utilise la softmax en sortie :

$$\hat{y}^{(k)} = \frac{\exp(w_{(L)}^T(k) z_{(L-1)})}{\sum_{j=1}^C \exp(w_{(L)}^T(j) z_{(L-1)})}$$

$w_{(L)j}$: column of $W_{(L)}$

combinée à la cross-entropie :

$$R(W) := -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_i^{(k)} \log \hat{y}_j^{(k)}$$

- TensorFlow Playground Regression Demo: [Voir](#)
- MLP 2D Visualization Demo: [Voir](#)

3.5.7 Lecture 10 — Deep Learning (Part 2)

- Neural Networks and Deep Learning (Michael Nielsen): [Voir](#)
- CNN 2D Visualization Demo: [Voir](#)
- ConvNetJS Visualization Demo: [Voir](#)
- Semantic Segmentation Demo: [Voir](#)

CHAPTER 4

Unsupervised ML

4.1 Dimensionality reduction

4.1.1 Definition

Dimensionality Reduction

Given high-dimensional samples $x_i \in \mathbb{R}^D$, find a low-dimensional representation $y_i \in \mathbb{R}^d$ with $d < D$.

4.1.2 Example

Image vectorization

$$x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3} = \mathbb{R}^{572448}$$

Even though the image lives in a 572 448-dimensional space, the true degrees of freedom are much fewer.

Donc on a compris que pour traiter une image c'est troooooo couteux, donc on va réduire sa dimension. Un autre exemple, si on prend un anneau en 3D, on ne perd que très peu d'informations en le regardant uniquement de face (comme en 2D). C'est notre motivation, on aura moins de dimensions à traiter, donc plus calcul plus rapide et plus faisable.

- **Principal Component Analysis (PCA)**: simple rapide et interprétable, bien pour les structures linéaires
- **Kernel PCA (KPCA)**: gérer les structures non linéaires, mais couteux en calcul mais il faut un bon kernel
- **Autoencoders**: plus flexibles et capturent des relations complexes, plus compliquer à entraîner et moins interprétable

4.2 Principal Component Analysis (PCA)

4.2.1 Definition

PCA Projection

$$y_i = W^T(x_i - \bar{x}), \quad W^T W = I_d, \quad \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

4.2.2 PCA Objective

Variance of the j -th projected dimension:

$$\text{var}(\{y_i^{(j)}\}) = \frac{1}{N} \sum_{i=1}^N \left(y_i^{(j)} - \bar{y}^{(j)} \right)^2$$

PCA Objective

Le PCA "cherche le point de vue" qui décrit le plus, donc où les données sont le plus éloignées, nos données. Si on reprend l'exemple de l'anneau, en le voyant de profil on voit une droite (très mauvaise interprétation), mais de face, on voit un cercle (excellente réduction de dimension)

4.2.3 1D PCA

Projection vector: $w_{(1)} \in \mathbb{R}^D$, $w_{(1)}^T w_{(1)} = 1$

Projection: $y_i = w_{(1)}^T (x_i - \bar{x})$

Projected mean:

$$\bar{y} = \frac{1}{N} \sum_{i=1}^N w_{(1)}^T (x_i - \bar{x}) = w_{(1)}^T \underbrace{\left(\frac{1}{N} \sum_{i=1}^N x_i \right)}_{\bar{x}} - w_{(1)}^T \bar{x} = 0$$

4.2.4 Covariance Matrix**Sample Covariance Matrix**

$$C = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T \in \mathbb{R}^{D \times D}$$

Variance after projection:

$$\text{var}(\{y_i\}) = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2 = \frac{1}{N} \sum_{i=1}^N \left(w_{(1)}^T (x_i - \bar{x}) \right)^2 = w_{(1)}^T C w_{(1)}$$

4.2.5 Optimization Problem

$$\max_{w_{(1)}} w_{(1)}^T C w_{(1)} \quad \text{s.t.} \quad w_{(1)}^T w_{(1)} = 1$$

Lagrangian:

$$L(w_{(1)}, \lambda_1) = w_{(1)}^T C w_{(1)} + \lambda_1 (1 - w_{(1)}^T w_{(1)})$$

Stationarity condition:

$$C w_{(1)} = \lambda_1 w_{(1)}$$

Multiplying left by $w_{(1)}^T$: $w_{(1)}^T C w_{(1)} = \lambda_1$, so projected variance = λ_1 .

1D PCA Solution

$w_{(1)}$ is the eigenvector of C with the **largest** eigenvalue λ_1 . The maximum projected variance equals λ_1 .

4.2.6 Higher Dimensional PCA

$$W = \begin{bmatrix} w_{(1)} & w_{(2)} & \cdots & w_{(d)} \end{bmatrix} \in \mathbb{R}^{D \times d}, \quad W^T W = I_d$$

$$\lambda_1 \geq \cdots \geq \lambda_d$$

Columns $w_{(1)}, \dots, w_{(d)}$ are the d eigenvectors of C with the d largest eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d$.

Projection: $y_i = W^T (x_i - \bar{x}) \in \mathbb{R}^d$

4.2.7 Limit Case: $d = D$

No dimensionality reduction

If all dimensions are kept:

$$d = D$$

- No dimensionality reduction
- No loss of information
- In 3D, PCA can be interpreted as a rotation of the data
- The dimensions in the new space are uncorrelated

4.2.8 Keeping Non-zero Eigenvalues

Intrinsic low dimensionality

Another option is to keep all eigenvectors corresponding to non-zero eigenvalues.

- The data is then truly low-dimensional
- Often occurs when $N < D$
- The resulting representations satisfy $d < D$
- No information is lost

4.2.9 Variance Retention

Choose d to retain at least 90% of the variance:

$$\sum_{j=1}^d \lambda_j \geq 0.9 \sum_{k=1}^D \lambda_k$$

4.2.10 Reconstruction

$$\hat{x}_i = \bar{x} + W y_i = \bar{x} + W W^T (x_i - \bar{x})$$

Reconstruction error: $e_i = \|\hat{x}_i - x_i\|^2$

PCA minimizes reconstruction error

The corresponding rectangular matrix W is the orthogonal matrix that minimizes the reconstruction error.

4.2.11 PCA Mapping

$$\hat{x} = \bar{x} + W y, \quad y \in \mathbb{R}^d$$

This mapping constrains $\hat{x}$ to lie in a subspace, and thus provides a form of regularization.

4.3 Kernel PCA

4.3.1 Feature Mapping

Map inputs to a high-dimensional feature space, then perform PCA in feature space.

$$x_i \mapsto \phi(x_i), \quad k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

4.3.2 Centered Feature Space

Assume centered features: $\frac{1}{N} \sum_{i=1}^N \phi(x_i) = 0$

Covariance matrix in feature space:

$$C = \frac{1}{N} \sum_{i=1}^N \phi(x_i) \phi(x_i)^T, \quad Cw_{(1)} = \lambda_1 w_{(1)}$$

4.3.3 Representer Theorem

Representer Theorem

The eigenvectors of C lie in the span of the mapped data:

$$w_{(1)} = \sum_{j=1}^N a_j \phi(x_j)$$

Substituting leads to an eigenvalue problem on the kernel matrix $K \in \mathbb{R}^{N \times N}$, $K_{ij} = k(x_i, x_j)$:

$$Ka = \lambda_1 Na$$

where $a \in \mathbb{R}^N$ and K is the kernel matrix.

4.3.4 Kernel PCA Projection

$$y_i = \phi(x_i)^T w_{(1)} = \sum_{j=1}^N a_j \underbrace{\phi(x_i)^T \phi(x_j)}_{k(x_i, x_j)} = \sum_{j=1}^N a_j k(x_i, x_j)$$

Everything is expressed in terms of kernel evaluations — ϕ need not be computed explicitly.

4.3.5 Non-centered Data

Centered features:

$$\tilde{\phi}(x_i) = \phi(x_i) - \frac{1}{N} \sum_{j=1}^N \phi(x_j)$$

Centered kernel matrix:

$$\tilde{K} = K - \mathbf{1}_N K - K \mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N, \quad \mathbf{1}_N = \frac{1}{N} \mathbf{1} \mathbf{1}^T, \quad \tilde{K}_{i,j} = \tilde{\phi}(x_i)^T \tilde{\phi}(x_j)$$

4.3.6 Kernel PCA vs PCA

Kernel PCA limitation

As PCA, Kernel PCA defines a mapping from the high-dimensional space to the low-dimensional one.

However, unlike PCA, Kernel PCA does not provide a mapping from the low-dimensional space back to the high-dimensional one.

4.4 Autoencoders

4.4.1 PCA Linear Mapping

$$y = W^T(x - \bar{x}), \quad \hat{x} = \bar{x} + Wy$$

4.4.2 Simple Autoencoder

Simple Autoencoder

$$\hat{x} = Wf(W^T x)$$

where $f(\cdot)$ is a nonlinear activation function.

4.4.3 Autoencoder: General Form

Autoencoder

Encoder: maps the input to a latent representation

Decoder: maps the latent representation to the output

Goal:

$$\hat{x} \approx x$$

4.4.4 Deep Autoencoder

Deep Autoencoder

Stacked layers:

$$z^{(j)} = f^{(j)}(W_{(j)}^T z^{(j-1)})$$

4.4.5 Training

Autoencoder Training

Objective: minimize reconstruction error

$$R(\{W_{(j)}\}) = \frac{1}{N} \sum_{i=1}^N \left\| \hat{x}_i(\{W_{(j)}\}) - x_i \right\|^2$$

- Non-convex optimization
- Solved with stochastic gradient descent

4.4.6 Dimensionality Expansion

Dimensionality Expansion

Because the mapping is nonlinear, the latent representation does not need to be low-dimensional.

This allows dimensionality expansion.

4.4.7 Denoising Autoencoder

Denoising Autoencoder

Idea: add noise to the input, train to reconstruct the clean version.
Forces the encoder to learn a robust representation rather than the identity.

4.4.8 Sparse Autoencoder

Sparse Autoencoder

Add an ℓ_1 sparsity penalty on the latent code $y_i \in \mathbb{R}^M$:

$$E_r(\{W_{(j)}\}) = \sum_{i=1}^N \sum_{k=1}^M |y_i^{(k)}|$$

Encourages most latent units to be zero for any given input.

4.4.9 Applications

Autoencoder Applications

- Compact representation / compression
- Retrieval and nearest-neighbor search in latent space

4.4.10 Autoencoder Mapping

Decoder mapping

The decoder defines a mapping from the latent space to the output space.
New samples can therefore be generated by decoding arbitrary latent vectors.

4.5 K-Means Clustering

Clustering

An unsupervised task: partition N unlabelled samples into groups such that

- points within the same cluster are similar,
- points in different clusters are dissimilar.

K-Means Problem

Given $\{x_i\}_{i=1}^N \subset \mathbb{R}^D$ and K the number of clusters K group the samples into K different groups.

Note: In the idea of clusters, the distance between points within a cluster is small, and between clusters is big.

K-Means Algorithm

1. **Initialize** $\mu_1, \dots, \mu_K$ (randomly).
2. **While not converged**
 - **Step 2.1: Assign each point x_i to the nearest center μ_k**
 - For each point x_i , compute the Euclidean distance to every $\{\mu_1, \dots, \mu_K\}$
 - Find the smallest distance
 - The point is said to be assigned to the corresponding cluster (note that each point is assigned to a single cluster)
 - **Step 2.2: Update each center μ_k based on the points assigned to it**
 - Recompute each center μ_k as the mean of the points that were assigned to it
 - **Convergence:**
 - The algorithm iteratively updates the cluster assignment for each point and the cluster centers. We need to eventually stop iterating
 - This could be achieved by a fixed number of iterations. However, setting this number is arbitrary and a too small number can lead to bad results
 - The algorithm is guaranteed to converge to a stable solution, where the cluster centers and the point assignments are fixed, i.e., do not change as more iterations are performed
 - The difference in assignments or center locations between two iterations can be used as criteria to stop the algorithm
 - **Importantly:** While the algorithm always converges, it does not always converge to the best (desired) solution
 - The solution depends on the center initialization

Importance of Feature Scaling

K-Means uses Euclidean distance, so features with large numerical ranges dominate. Always **standardize** features (zero mean, unit variance) before clustering, or use a distance function appropriate to the data type.